

Arquitectura y Especificaciones de la Plataforma IIP

Juan José Martínez Guerrero
LABCapital — Laboratorio de Innovación Pública
Veeduría Distrital
Bogotá, Colombia
labcapital@veeduriadistrital.gov.co

Abstract—La plataforma del Índice de Innovación Pública (IIP) constituye un ecosistema de gestión de datos de alto rendimiento, diseñado como una arquitectura de microservicios orientada al dominio (DDD) para servir como el motor central de inteligencia y trazabilidad de la Veeduría Distrital. El sistema implementa una arquitectura basada en contenedores Docker que integra componentes especializados: autenticación, lógica central (Core), persistencia (Alembic/Seeders), almacenamiento persistente en PostgreSQL y un proxy inverso Nginx. Desarrollada bajo el patrón Fast API con capas de servicios, unidades de trabajo (UOW) y repositorios, la solución centraliza el ciclo de vida completo de las versiones 2019, 2021, 2023 y 2025 del IIP, abarcando desde la gestión de cuestionarios y respuestas hasta el procesamiento analítico de resultados. Esta infraestructura está diseñada como una solución escalable y abierta, preparada para la integración futura con motores de vectorización, sistemas de Generación Aumentada por Recuperación (RAG) y protocolos de Model Context Protocol (MCP), facilitando tanto la captura de nuevas propuestas y la gestión de procesos de evaluación, como la democratización de datos a través de portales de datos abiertos.

Index Terms—Public Innovation Index, Domain-Driven Design, Microservices Architecture, Data Governance, FastAPI, Scalable Systems, Enterprise Application Architecture, Public Policy Informatics

I. INTRODUCCIÓN

La plataforma del índice de Innovación Pública (desde ahora IIP) se erige como un sofisticado ecosistema tecnológico, concebido como un hub de datos multidominio de alta disponibilidad, diseñado específicamente para satisfacer las necesidades analíticas y de gobernanza de la Veeduría Distrital. Este sistema no solo actúa como un repositorio centralizado, sino como un motor de procesamiento inteligente capaz de orquestar la complejidad inherente a los datos distritales, integrando de manera fluida la gestión dinámica de formularios, la administración de actores estratégicos y sistemas de evaluación robustos.

Bajo una arquitectura de microservicios estrictamente desacoplada, la plataforma garantiza una separación de responsabilidades que optimiza el ciclo de vida del software, permitiendo que los servicios de autenticación, la lógica de negocio central y la capa de persistencia operen como entidades independientes, aunque perfectamente cohesionadas. Todo este desarrollo está consolidado bajo una estrategia de monorepo, la cual permite la gestión unificada del código

fuente, facilitando el intercambio de lógica a través de una biblioteca interna compartida. Esta infraestructura técnica, robusta y escalable, ha sido diseñada con una visión a largo plazo, garantizando que el sistema sea capaz de evolucionar desde una solución de gestión administrativa hacia un núcleo tecnológico preparado para la integración de sistemas de vectorización, arquitecturas RAG, y protocolos de comunicación de última generación como MCP, consolidándose así como la infraestructura de datos definitiva para la innovación pública.

A. Propósito

El propósito fundamental de la plataforma IIP es democratizar el acceso y la gestión del conocimiento derivado del Índice de Innovación Pública (IIP), funcionando como la columna vertebral de datos para la Veeduría Distrital. El alcance del sistema abarca la consolidación histórica y analítica de todas las versiones del índice (2019, 2021, 2023 y 2025), transformando una estructura de datos fragmentada en un modelo de información coherente, versionable y auditable.

En términos operativos, la plataforma está diseñada para satisfacer tres pilares críticos:

- 1) Gestión Integral del Ciclo de Vida: Desde la captura de nuevas respuestas y la gestión de actores, hasta la evaluación técnica realizada por el colegio calificador.
- 2) Interoperabilidad de Datos: Actuar como fuente única de verdad para la publicación de datos abiertos, facilitando el consumo analítico externo y asegurando la transparencia gubernamental.
- 3) Extensibilidad Inteligente: Servir como base de datos para sistemas avanzados, incluyendo la futura integración de servicios de RAG (Generación Aumentada por Recuperación) y agentes de IA, permitiendo consultas semánticas complejas sobre todo el histórico de resultados del IIP.

B. Alcance

El presente documento pretende servir al lector como la fuente de verdad sobre el funcionamiento hasta la fecha de redacción del presente documento, de la plataforma API de gestión de datos del Índice de Innovación Pública.

Teniendo eso en cuenta, el documento no contempla desglosar el funcionamiento de otras herramientas usadas alrededor o en simultáneo / paralelo con la API que aquí se documenta. Entre estas últimas encontramos front ends de consumo de los

datos, dashboards, el aplicativo de captura del instrumento, los sistemas de trabajadores para analíticas de datos o el acceso por API de agentes externos a la propia plataforma.

C. Audiencia destinada

La plataforma IIP está diseñada para servir a un ecosistema diverso de usuarios, cada uno con necesidades específicas de interacción y niveles de acceso diferenciados:

- **Entidades Distritales (Sujetos de Evaluación):** Son los responsables de la carga de información. A través de funcionarios autorizados, representan a sus entidades para responder al IIP. El sistema garantiza que esta escritura sea trazable, auditable y centralizada, convirtiéndose en su herramienta oficial de reporte ante la Veeduría.
- **Niveles de Decisión Estratégica (Secretaría General y Cabezas de Sector):** Estos actores utilizan los resultados del IIP como el principal insumo de diagnóstico. Su acceso está enfocado en la analítica de alto nivel para identificar brechas, priorizar inversiones y dirigir recursos hacia áreas que requieren mejoras sustanciales en sus capacidades de innovación pública.
- **Ecosistema de Innovación (Labs Distritales como iBo):** Estos laboratorios actúan como catalizadores. Utilizan la data cruda y los resultados procesados para diseñar intervenciones, experimentos y estrategias de acompañamiento técnico a las entidades, transformando los puntajes bajos en planes de mejora concretos.
- **Organismos de Control:** Utilizan el sistema como la fuente de verdad técnica para la generación de recomendaciones, auditorías y el seguimiento a políticas públicas transversales, incluyendo el cumplimiento del CONPES 04 y otros marcos normativos vigentes.
- **Comunidad Académica, Observatorios y ONGs:** Actores que buscan democratizar el dato. Acceden a los resultados mediante portales de visualización, herramientas de exportación y consultas semánticas mediante IA, fomentando el escrutinio público y la investigación longitudinal.
- **Arquitectos y Desarrolladores:** Personal técnico encargado de la integración del IIP con sistemas externos, servicios de RAG o protocolos de comunicación MCP, garantizando que el sistema evolucione como una pieza fundamental de la infraestructura digital del Distrito.

D. Visión general de la plataforma

La plataforma IIP se posiciona como el ecosistema tecnológico de referencia para la gestión, procesamiento y democratización de los datos asociados al Índice de Innovación Pública de Bogotá. Más allá de actuar como un repositorio estático, la plataforma se ha concebido bajo tres pilares fundamentales que transforman la forma en que el Distrito gestiona su innovación:

- 1) **Centralización de la Fuente de Verdad:** Históricamente, la información del IIP se encontraba dispersa en estructuras no normalizadas. La plataforma IIP consolida todas las versiones históricas (2019-2025) en un modelo de datos único, auditable y versionable, garan-

tizando que tanto los organismos de control como los tomadores de decisiones operen sobre la misma base de datos íntegra.

- 2) **Motor de Inteligencia y Trazabilidad:** La arquitectura implementa un flujo transaccional donde cada interacción (desde la respuesta inicial de una entidad hasta la calificación final por parte del colegio calificador) es registrada y validada. Este nivel de trazabilidad permite no solo la transparencia, sino también la creación de analíticas complejas que identifican patrones de mejora en tiempo real.
- 3) **Ecosistema Abierto y Escalable:** La plataforma está diseñada para trascender su función administrativa. Al exponer la data cruda a través de APIs documentadas, permite que laboratorios como iBo, investigadores académicos y herramientas de IA (RAG/MCP) se integren de manera nativa. Esto asegura que el sistema no sea una “caja negra”, sino un motor abierto que se adapta tanto a los reportes de política pública (como el CONPES 04) como a las necesidades de análisis de datos a futuro.

En esencia, la plataforma IIP es la infraestructura habilitante que convierte la complejidad administrativa en evidencia técnica, facilitando que las entidades, los laboratorios de innovación y los entes de control no solo midan, sino que transformen el desempeño del sector público en Bogotá.

E. Principios de diseño

La arquitectura de la plataforma IIP ha sido fundamentada sobre cinco principios rectores que garantizan la integridad, escalabilidad y transparencia del ecosistema:

- 1) **Desacoplamiento de Dominios (Arquitectura Hexagonal/DDD):** La lógica de negocio reside en una capa central pura, independiente de frameworks web o bases de datos específicas. Esto permite que el sistema evolucione (cambiar una base de datos o migrar a una nueva versión de Python) sin alterar las reglas de validación del Índice de Innovación Pública.
- 2) **Consistencia como Fuente de Verdad:** El sistema opera bajo el principio de centralización de la persistencia. Al utilizar SQLAlchemy y Alembic, garantizamos que cualquier dato, desde un histórico de 2019 hasta una nueva respuesta, mantenga la integridad relacional. No existen datos duplicados ni versiones divergentes de la realidad.
- 3) **Reproducibilidad Total (Infraestructura como Código):** El despliegue no debe depender de configuraciones manuales (“serendipia de servidor”). Mediante el script `launcher.sh`, el sistema garantiza que cualquier entorno (ya sea desarrollo o producción) pueda levantarse desde cero con una configuración consistente, incluyendo esquemas y semillas de datos.
- 4) **Seguridad por Diseño:** La confianza es crítica para una herramienta de la Veeduría. La seguridad no se añade al final; se implementa mediante capas: autenticación JWT con firma ED25519, hashing de contraseñas con Argon2, y gestión de secretos aislada para evitar la exposición de credenciales en el repositorio.

- 5) **Apertura para la Extensibilidad (Data-First):** El diseño anticipa el futuro. La plataforma no está cerrada; se construye con el principio de “API-first”, permitiendo que sistemas externos, ya sean herramientas de visualización, agentes de IA para análisis semántico (RAG) o protocolos de integración (MCP), consuman los datos de forma estructurada y controlada sin comprometer la seguridad.

II. ARQUITECTURA DEL SISTEMA

La plataforma IIP adopta un diseño modular basado en microservicios, eliminando la complejidad de los monolitos tradicionales mediante una estrategia de monorepo gestionada por workspaces. Esta configuración facilita el escalado horizontal mediante la puesta en marcha rápida de réplicas de servicios ante picos de demanda. La arquitectura está diseñada para maximizar la independencia de los dominios funcionales (Autenticación, Lógica de Negocio y Persistencia), asegurando al mismo tiempo una gobernanza centralizada del código. La infraestructura se apoya en la contenerización para garantizar la portabilidad y una escalabilidad granular, permitiendo que cada componente evolucione de manera autónoma sin comprometer la integridad del ecosistema.

Asimismo, la arquitectura apuesta por la limpieza del código, la no repetición de utilidades y la predictibilidad (principios comúnmente asociados a KISS y DRY). Paralelamente, se busca que el código responda de manera orgánica a las necesidades del Índice, evitando forzarlo a adaptarse a patrones rígidos de desarrollo. En consecuencia, el patrón de diseño implementado sigue las directrices generales de DDD [1], pero se toma ciertas libertades estratégicas para garantizar la estricta separación de responsabilidades, la practicidad operativa y la seguridad.

A. Arquitectura de alto nivel

A nivel técnico, el sistema se organiza bajo el principio de separación de responsabilidades, utilizando una arquitectura de capas basada en Domain-Driven Design (DDD). Esta estructura se divide en dos planos fundamentales: el despliegue de infraestructura y la organización lógica del código.

En el plano operativo, un proxy inverso Nginx actúa como la única puerta de entrada hacia la plataforma. Este componente gestiona el enrutamiento seguro de las peticiones del usuario hacia los servicios correspondientes (Auth para identidad y Core para negocio), los cuales interactúan con un motor centralizado de PostgreSQL. Por su parte, el servicio de Persistence opera como un componente de gestión fuera del flujo crítico de usuario, dedicado exclusivamente a la integridad del ciclo de vida de los datos (migraciones, poblamiento y respaldos), tal como se detalla en Fig. 1.

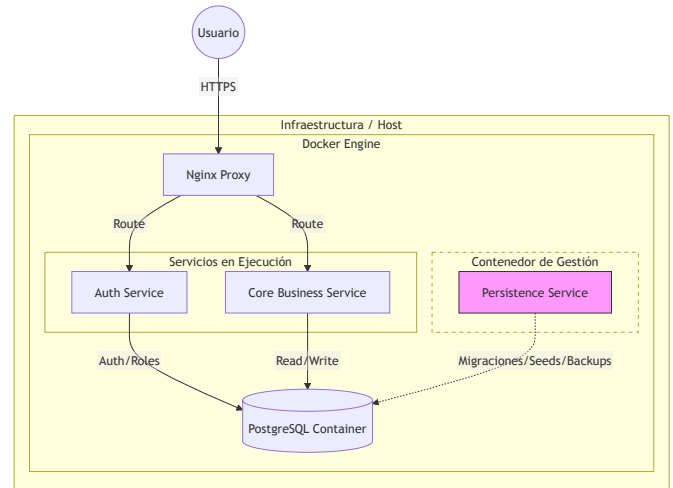


Fig. 1. Arquitectura de despliegue.

En el plano de desarrollo, el uso de un monorepo nos permite que servicios independientes (como Auth, Core y Persistence) compartan una capa de utilidades y modelos definida en el módulo Shared. Como se ilustra en Fig. 2, esta dependencia garantiza una consistencia semántica en todo el sistema y evita la duplicidad de lógica, permitiendo que las entidades de dominio sean coherentes en toda la plataforma.

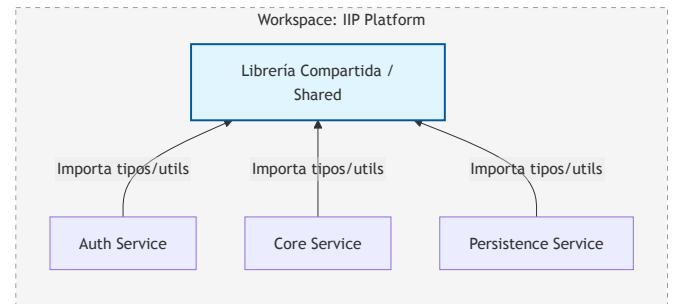


Fig. 2. Dependencias de módulos dentro del monorepo.

El sistema se despliega mediante una arquitectura basada en contenedores Docker, orquestada para garantizar el aislamiento y la escalabilidad de cada componente. Un proxy inverso Nginx actúa como puerta de enlace, gestionando el enrutamiento del tráfico hacia los servicios correspondientes y asegurando una comunicación segura. La lógica interna sigue el patrón de diseño Domain-Driven Design (DDD), implementando una arquitectura de capas que organiza el código en servicios, unidades de trabajo (UOW) y repositorios, asegurando que la lógica de negocio permanezca desacoplada de los detalles técnicos de persistencia. Esta disposición permite una evolución independiente de cada módulo, desde la capa de autenticación hasta el motor de procesamiento de datos gestionado por el servicio de core y el motor de base de datos PostgreSQL.

B. Stack tecnológico

El sistema está construido sobre un ecosistema de software moderno, seleccionado específicamente para optimizar el

rendimiento asíncrono, la seguridad criptográfica y la manipulación eficiente de datos:

- 1) **Ecosistema Base:** Python 3.12.13 administrado mediante `uv`, aprovechando el soporte de `workspaces` nativo para coordinar el monorepo y la inyección local de dependencias compartidas (`Packages/shared`).
- 2) **Framework Web:** `FastAPI` junto con `Uvicorn`, operando de forma 100% asíncrona para maximizar la concurrencia.
- 3) **Persistencia Relacional:** `PostgreSQL 18` gestionado a través de `SQLAlchemy 2.0` (ORM) y el driver de alto rendimiento `asyncpg` para operaciones no bloqueantes. Las mutaciones del esquema se controlan mediante `Alembic`.
- 4) **Caché y Estado Volátil:** `Valkey` (bifurcación de alto rendimiento de Redis) como base de datos en memoria para la gestión de sesiones y almacenamiento en caché.
- 5) **Procesamiento de Datos:** `Pandas` y `NumPy` para análisis computacional masivo, complementados con `OpenPyXL` para la ingesta y exportación estructurada de archivos Excel.
- 6) **Seguridad Avanzada:** Hashing de credenciales mediante `Argon2` (ganador del Password Hashing Competition) y firmas criptográficas `ED25519` para tokens JWT. Adicionalmente, implementa **cifrado simétrico a nivel de aplicación (Field-Level Encryption)** mediante el protocolo `Fernet` (AES-128-CBC + HMAC-SHA256 de la suite `cryptography`) para asegurar datos altamente sensibles en reposo antes de persistirlos en la base de datos.
- 7) **Cliente y Red:** `HTTPX` para peticiones asíncronas entre microservicios (comunicación inter-container) y `aiofiles` para la manipulación asíncrona del sistema de archivos (`/uploads`).

C. *Arquitectura de despliegue (Docker)*

La plataforma implementa una arquitectura basada en microservicios orquestada mediante Docker Compose, diseñada para separar responsabilidades, aislar el tráfico de red y gestionar de manera segura los datos sensibles.

- **nginx:** Basado en `nginx:stable-alpine`. Opera como proxy inverso y terminal TLS, enrutando el tráfico HTTP/HTTPS externo hacia los contenedores de la aplicación.
- **db:** Motor relacional `postgres:18` en el que se almacena la información persistente del aplicativo (excluyendo archivos físicos y datos volátiles). Garantiza la persistencia de datos mediante el volumen nombrado `postgresql`.
- **cache:** Instancia de `valkey:alpine` que provee almacenamiento en memoria para gestionar la información volátil de sesiones y mitigar la carga de peticiones a la base de datos.
- **base:** Imagen base construida localmente (`./Dockerfile`) como `app-base` para consolidar y precompilar las dependencias comunes del monorepo.

- **auth:** Microservicio en `FastAPI` que centraliza la gestión de identidad, autenticación y autorización mediante esquemas RBAC y ReBAC.
- **core:** Microservicio en `FastAPI` que concentra la lógica principal de negocio y el procesamiento asociado a los formularios y sus respuestas.
- **persister:** Contenedor de administración encargado de ejecutar las migraciones de Alembic, los scripts de poblado y las copias de seguridad mediante el montaje de volúmenes del anfitrión.

TABLE I
COMPONENTES DEL SISTEMA Y SUS RESPONSABILIDADES.

Servicio	Responsabilidad	Entidades Clave
Nginx	Proxy inverso y enrutamiento	Configuración, Certificados
Auth	Gestión de identidad y seguridad	User, Role, RefreshSession
Core	Lógica de negocio y procesamiento de dominio	Form, Submission, Actor, Grade
Persistence	Ciclo de vida de BD, migraciones y carga	Alembic, Seeder, Populator
Shared	Lógica común, modelos ORM y utilidades	Base, AccessContext, Hashing

D. *Variables de entorno*

La plataforma implementa un esquema de configuración centralizado que desacopla el comportamiento del entorno mediante variables de entorno, aplicando valores por defecto (**sane defaults**) para permitir el arranque inmediato en entornos locales.

- **Inyección de Variables y Requisitos Mínimos:** Únicamente las variables `PUBLIC_ORIGINS` y `PRIVATE_ORIGINS` son estrictamente requeridas en despliegues reales. Parámetros operativos como puertos (`PORT_AUTH=8000`, `PORT_CORE=8001`), credenciales de base de datos, tiempo de vida de tokens JWT (15 minutos para acceso y 7 días para refresco) y niveles de log (`LOGLEVEL=error`) cuentan con valores preconfigurados en la aplicación.
- **Políticas Dinámicas de CORS y Seguridad:** El parámetro `PRODUCTION_MODE` altera automáticamente las directivas de seguridad web en tiempo de ejecución:
 - **Modo Desarrollo** (`PRODUCTION_MODE=false`): Habilita comodines (`*`) para orígenes, métodos, cabeceras y hosts permitidos, simplificando la integración local.
 - **Modo Producción** (`PRODUCTION_MODE=true`): Convierte

te las cadenas separadas por comas de `PUBLIC_ORIGINS` y `PRIVATE_ORIGINS` en listas explícitas de orígenes permitidos. Restringe los métodos HTTP a `GET`, `POST`, `PUT`, `DELETE` y `OPTIONS`, y valida un conjunto estricto de cabeceras autorizadas (`Authorization`, `Content-Type`, `Cookie`, `X-Platform`, entre otras).

E. Arquitectura de volúmenes

El sistema utiliza una estrategia mixta de persistencia basada en volúmenes nombrados (para datos estructurados y archivos compartidos) y montajes de tipo bind (bind-mounts) para configuraciones y sincronización de código en tiempo de ejecución.

a) 1. Volúmenes Nombrados (Named Volumes):

- **postgresql (mapeado como pg):** Garantiza la persistencia del estado físico de la base de datos PostgreSQL, montado en `/var/lib/postgresql` dentro del contenedor `app_db`.
- **app_uploads (mapeado como uploads):** Volumen compartido entre los servicios de la aplicación (`auth`, `core` y `nginx`). Centraliza los archivos cargados por los usuarios y los expone en la ruta `/uploads` de cada contenedor.

b) 2. Montajes de Directorio (Bind-Mounts):

- **Copias de seguridad (/backups):** Sincroniza el directorio local `./Persistence/src/backups` de forma bidireccional entre `app_db` (generación de volcados) y `app_persist` (gestión o extracción de respaldos).
- **Ciclo de vida de Base de Datos (persist):**
 - Mapea las migraciones de base de datos desde `./Persistence/src/migrator/.alembic/versions`
 - Mapea scripts de extracción de datos en `./Persistence/src/scripts/.extracted`
- **Configuración del Servidor:** Monta el archivo maestro de configuración `./Secrets/nginx.conf` en modo de solo lectura (`ro`) dentro del contenedor `app_nginx`.

F. Gestión de secretos

La gestión de credenciales sensibles (`postgres_password`, `valkey_password`, llaves JWT `ED25519`, tokens de GitHub y certificados SSL) utiliza el mecanismo nativo de `secrets` de Compose desde `./Secrets`, evitando la exposición de claves en variables de entorno de texto plano.

Aunque la plataforma puede leer parámetros directamente del entorno, se recomienda inyectar credenciales sensibles mediante el sistema nativo de secretos de Docker Compose. Para el control de versiones seguro, la arquitectura integra **Mozilla SOPS** (Secrets OPerationS), permitiendo mantener los archivos de secretos cifrados en el repositorio y descifrarlos dinámicamente solo para el servicio que los solicita. La modificación del archivo base `users.toml` es un requi-

sito indispensable en producción para revocar las contraseñas predeterminadas de los usuarios administradores.

Para evitar la sobrecarga de gestionar secretos en ejecuciones locales sin contenedores, el proyecto provee un archivo de configuración `.envrc` compatible con `direnv`. Este mecanismo detecta el directorio de trabajo y exporta automáticamente los secretos e indicadores de entorno como variables locales, acelerando la iteración de desarrollo sin comprometer las políticas de producción.

G. Comunicación entre servicios

El aislamiento y la interconexión entre los contenedores de la plataforma se rigen por tres principios de red:

- **Aislamiento y Exposición Pública:** La red `app_nginx_network` es el único vector expuesto hacia el exterior a través del mapeo de puertos en el anfitrión (`8000` y `8001`). Por el contrario, `app_db_network` y `app_cache_network` son redes internas tipo `bridge` estrictamente aisladas; ni PostgreSQL ni Valkey exponen puertos hacia el host, impidiendo cualquier intento de conexión directa desde fuera de la red virtual de Docker.
- **Pila de Protocolos y Transporte:** La comunicación entre los componentes opera sobre la pila TCP/IP interna gestionada por el daemon de Docker, utilizando el protocolo HTTP/1.1 para la transferencia de peticiones entre el proxy Nginx y los microservicios `Auth` y `Core`.
- **Naturaleza del Tráfico (Stateless TCP):** Todas las interacciones de la API siguen un esquema transaccional solicitud-respuesta. La arquitectura omite intencionalmente protocolos de transmisión en tiempo real o persistentes (como WebSockets o gRPC), garantizando un modelo puramente **stateless** sobre conexiones TCP de corta duración que facilita la escalabilidad y reduce el consumo de memoria en los sockets.

El servidor Nginx actúa como API Gateway y punto de terminación TLS de la plataforma, aislando la red pública de los contenedores de aplicación.

- **Paridad de Puertos con Compose:** Mantiene estricta correspondencia con las variables definidas en `compose.yaml` y los Dockerfiles (`PORT_AUTH=8000` y `PORT_CORE=8001`). Los puertos de escucha SSL del proxy reflejan exactamente los puertos internos expuestos por cada microservicio.
- **Gestión de Certificados TLS:** Carga los pares de claves (`https_public` y `https_private`) inyectados en `/run/secrets/`. Estos certificados son autogenerados de forma local mediante el script automatizado de gestión de secretos de la plataforma para entornos de desarrollo/pruebas, o bien provistos por el administrador mediante certificados de producción emitidos por autoridades como Let's Encrypt.
- **Enrutamiento por DNS Interno:** Utiliza la resolución de nombres nativa de Docker mediante `proxy_pass` para redirigir peticiones desde el puerto `8000` hacia `http://auth:8000` y desde el puerto `8001` hacia `http://core:8001`. Estos puertos pueden ser sobrescritos simplemente definiendo valores en

el archivo `.env` y recompilando las imágenes con `docker compose build base && docker compose build`

- **Preservación del Contexto de Red:** Transfiere los encabezados `Host`, `X-Real-IP`, `X-Forwarded-For` y `X-Forwarded-Proto` (`https`), permitiendo a los microservicios validar la IP real del cliente y reconocer el esquema HTTPS original para la emisión de tokens y directivas de seguridad.

TABLE II

MAPEO DE REDES, AISLAMIENTO Y CONECTIVIDAD ENTRE SERVICIOS DEL SISTEMA.

Servicio	Redes Asignadas	Conectividad / Accesos Directos	Exposición de Puertos
nginx	app_nginx	Acceso directo a auth y core	Público (80, 443)
auth	app_nginx, app_db_net, app_cache_net	Acceso a db y cache. Accesible por nginx y por core	Interno (8000)
core	app_nginx, app_db_net, app_cache_net	Acceso a db, cache. Accesible por nginx y por auth	Interno (8000)
persist	app_db_net	Acceso directo a db para migraciones y pobladores	Ninguno (Tarea/CLI)
cache	app_cache_net	Acepta conexiones de auth y core	Aislado (6379 interno)
db	app_db_net	Acepta conexiones de auth, core y persist	Aislado (5432 interno)

H. Estrategia de construcción de imágenes

La compilación se estructura en dos niveles para maximizar el reúso de capas: una imagen base compartida (`app-base`) y tres imágenes derivadas para los servicios finales (`Auth`, `Core` y `Persistence`).

- **Imagen Base Compartida (`app-base`)**

El `Dockerfile` raíz aplica una construcción multietapa que primero aísla los archivos `pyproject.toml` e `__init__.py` de `Packages/shared` (`structural-setup`). En la siguiente etapa (`base-env`), instala librerías nativas como `libpq-dev` sobre Python 3.12 slim. Finalmente

(`final-app`), transfiere la implementación del paquete compartido e instala las dependencias de forma local sin conexión a red (`--no-deps`).

- **Optimización de Caché (Mock Layout)**

Para evitar la re-descarga constante de paquetes pesados (`FastAPI`, `Argon2`), las imágenes derivadas de `app-base` aplican un patrón en cuatro fases:

1. **Aislamiento:** Copia únicamente `pyproject.toml` del servicio para separar las dependencias del código fuente.
2. **Estructura Ficticia:** Genera carpetas vacías (`mkdir -p application ...`) y un archivo `main.py` temporal para que el empaquetador `Hatchling` pre-instale dependencias mediante `pip install .`
3. **Límite de Caché:** Transfiere el código fuente real, invalidando la caché del motor Docker únicamente al modificar archivos de la aplicación.
4. **Sincronización Local:** Ejecuta `pip install --no-deps .` para vincular los puntos de entrada locales en menos de un segundo sin consultar índices remotos.

Configuración de Contenedores Específicos

Los contenedores `app_auth` y `app_core` exponen los puertos `8000` y `8001` respectivamente, ejecutando `uvicorn main:api` para atender peticiones. En contraste, `app_persist` inyecta `PORT_AUTH` y `PORT_CORE` para interactuar internamente con los endpoints durante la carga de datos, operando sin puertos expuestos al exterior vía `/entrypoint.sh`. Si se desea cambiar los puertos asignados a los microservicios, basta con cambiar las variables de ambiente correspondientes como se menciona en [Sección II.D].

I. Librería compartida

III. ARQUITECTURA DE DOMINIO

La plataforma IIP adopta un diseño modular basado en microservicios, eliminando la complejidad de los monolitos tradicionales mediante una estrategia de monorepo gestionada por **workspaces**. Esta arquitectura está diseñada para maximizar la independencia de los dominios funcionales (Autenticación, Lógica de Negocio y Persistencia), asegurando al mismo tiempo una gobernanza centralizada del código. La infraestructura se apoya en la contenerización para garantizar la portabilidad y la escalabilidad granular, permitiendo que cada componente evolucione de manera autónoma sin comprometer la integridad del ecosistema.

La arquitectura se organiza bajo el principio de separación de responsabilidades, utilizando un diseño de capas basado en **Domain-Driven Design** (DDD). Esta estructura se divide en dos planos fundamentales: el despliegue de infraestructura y la organización lógica del código. En el plano operativo, el sistema garantiza que la lógica de negocio permanezca aislada de los detalles técnicos, mientras que el plano de desarrollo aprovecha el monorepo para compartir tipado y utilidades a través de una librería centralizada, garantizando una consistencia semántica absoluta en todo el sistema.

A. Diseño guiado por el dominio (DDD)

El servicio Core implementa una arquitectura basada en Domain-Driven Design [1] (DDD), organizada en capas que separan las responsabilidades de presentación, aplicación, dominio e infraestructura. Esta organización permite mantener la lógica de negocio desacoplada de los mecanismos de persistencia y comunicación, facilitando la mantenibilidad, la extensibilidad y las pruebas unitarias.

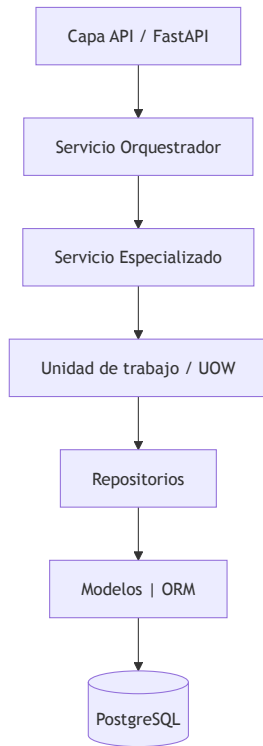


Fig. 3. Arquitectura de capas orientada al dominio (DDD).

- 1) **Capa API:** Expone los endpoints REST mediante FastAPI y actúa como punto de entrada al sistema. Su responsabilidad se limita a la validación de solicitudes, autenticación y serialización de respuestas.
- 2) **Servicio Orquestrador:** Coordina el flujo de ejecución de los casos de uso. Esta capa encapsula la lógica de alto nivel, delegando las operaciones específicas en servicios especializados sin contener reglas de negocio propias.
- 3) **Servicios Especializados:** Implementan la lógica de negocio asociada a cada dominio funcional, como la gestión de formularios, respuestas o auditorías. Estos servicios utilizan la Unidad de Trabajo para ejecutar operaciones transaccionales de forma consistente.
- 4) **Unidad de Trabajo y Repositorios:** La Unidad de Trabajo (UOW) administra el ciclo de vida de las transacciones y garantiza la atomicidad de las operaciones. Los repositorios abstraen el acceso a los datos, desacoplando la lógica de negocio de la tecnología de persistencia utilizada.
- 5) **Modelos ORM y Base de Datos:** La capa de persistencia emplea modelos definidos con SQLAlchemy

ORM para mapear las entidades del dominio a la base de datos PostgreSQL, proporcionando una interfaz orientada a objetos sobre el almacenamiento relacional.

Esta separación de responsabilidades facilita la evolución independiente de cada capa, mejora la mantenibilidad del sistema y simplifica la implementación de pruebas unitarias al reducir el acoplamiento entre la lógica de negocio y la infraestructura.

B. Ciclo de vida de la solicitud

El ciclo de vida de una petición sigue un flujo estricto para garantizar la seguridad:

- 1) **Ingreso:** La solicitud llega al Proxy Nginx, donde se ejerce el balanceo de carga y envío al host correspondiente dependiendo de la solicitud.
- 2) **Autenticación:** El middleware de FastAPI en el servicio Core decodifica y verifica la firma (ED25519) y la integridad del `AccessContext`.
- 3) **Autorización:** Se inyecta la dependencia `AccessContext` que valida si el `sub` y los `assignments` tienen permisos sobre el `entity_id` solicitado.
- 4) **Orquestación:** El Servicio de Aplicación recibe el comando y abre la **Unidad de Trabajo**.
- 5) **Persistencia:** La UOW delega en los **Repositorios** para recuperar o guardar entidades.
- 6) **Respuesta:** La UOW confirma la transacción y la capa API serializa el resultado hacia el cliente.

C. Desglose capa API

La capa de presentación constituye el límite exterior del sistema. Implementada sobre el marco de trabajo FastAPI, su responsabilidad exclusiva es gestionar el protocolo de transporte HTTP. Esto abarca la deserialización de peticiones, la validación formal de esquemas, la inyección de dependencias en tiempo de ejecución y la serialización de respuestas salientes, absteniéndose por completo de intervenir en la lógica de control transaccional o de negocio.

a) Aislamiento y Delegación de Casos de Uso:

Los controladores actúan bajo el patrón de controladores delgados (**thin controllers**). Ningún punto de enlace (**end-point**) interactúa de forma directa con la base de datos relacional, la infraestructura de caché o los modelos del dominio.

- **Inyección de Dependencias Decoupled:** Mediante la directiva `Depends()`, FastAPI resuelve e inyecta la factoría del servicio de aplicación (`get_auth_service`). Esto desacopla la firma de las funciones de la instanciación manual de las clases operativas, facilitando el aislamiento en pruebas unitarias mediante la sustitución por componentes simulados (**mocks**).
- **Contrato Limpio:** Las funciones interceptan el flujo de datos de entrada (`RequestRegister`, `RequestLogin`), delegan de inmediato la ejecución hacia el método correspondiente de `AuthAppService` y retornan estructuras estandarizadas.

darizadas de datos (`ResponseMessageSchema`, `ResponseAuthSchema`).

b) *Gestión del Contexto de Transporte de Sesión* (`SessionContext`):

Para evitar que los controladores manipulen directamente cabeceras crudas o muten variables de bajo nivel del protocolo HTTP, la arquitectura introduce la abstracción `SessionContext`. Inyectada como una dependencia unificada, se encarga de centralizar las interacciones con el estado de la conexión:

- 1) **Abstracción de Lectura:** Extrae de forma transparente el token de actualización de las peticiones entrantes (`ctx.refresh_token`), abstrayendo al punto de enlace de conocer si el valor viaja en cabeceras de autorización, variables de formulario o cookies seguras.
- 2) **Construcción Homogénea de Respuestas** (`make_response`): Modela y ensambla el cuerpo de la respuesta para credenciales asimétricas, encapsulando la lógica de inyección de metadatos o el establecimiento de cookies del lado del servidor.
- 3) **Mutación del Estado del Cliente** (`unset_refresh_cookie`): En casos de uso de cierre de sesión (`logout`), se encarga de emitir de forma segura las directivas de invalidación de cookies hacia el navegador o cliente HTTP, limpiando el rastro de la sesión fuera de los límites de la aplicación.

Seguridad en Tipado y Serialización:

El diseño aprovecha las capacidades de tipado estricto en el intercambio de datos mediante las siguientes directivas de infraestructura:

- **Protección de Criptas de Datos:** El acceso a cadenas sensibles (como contraseñas en texto plano) se gestiona mediante tipos especializados que exponen el método `.get_secret_value()`. Esto evita la fuga accidental de credenciales en trazas de registros de depuración (`logs`) o volcados de memoria del sistema.
- **Estrategia de Exclusión Dinámica:** La configuración `response_model_exclude_none=True` en los decoradores de rutas limpia las cargas útiles de salida. El sistema purga en tiempo de ejecución cualquier campo con valor nulo (`None`), minimizando el ancho de banda consumido en la red y garantizando contratos JSON estrictos y predecibles para los clientes de la API.

D. *Desglose capa orquestador*

La capa de Servicios de Aplicación (`AuthAppService`) actúa como el punto de entrada principal para la capa de presentación (controladores o puntos de enlace de la API). Su propósito fundamental es modelar los casos de uso puros del sistema, abstrayendo por completo a los controladores externos de la gestión transaccional y de la coordinación de los servicios internos.

a) *Gestión del Límite Transaccional:*

A diferencia de los servicios de dominio o de infraestructura, los servicios de aplicación tienen la responsabilidad exclusiva de abrir, controlar y cerrar las fronteras de la transacción mediante el uso del gestor de contexto de la Unidad de Trabajo (`async with AuthUoW() as uow:`).

- **Aislamiento Estricto:** Los controladores de la API no interactúan con el ciclo de vida de la base de datos ni conocen la existencia del objeto `uow`. El servicio de aplicación inicializa de forma segura el contexto transaccional al arrancar el caso de uso y asegura su cierre al finalizar.

- **Inyección del Contexto:** Una vez abierto el bloque de la Unidad de Trabajo, la instancia activa de `uow` se inyecta directamente como un parámetro en los métodos de los servicios subyacentes. Esto permite que múltiples servicios compartan la misma transacción exacta de forma transparente.

b) *Coordinación de Casos de Uso Multidominio:*

El servicio de aplicación no ejecuta lógica algorítmica ni cálculos de negocio. Su función es puramente de coordinación y delegación (Orquestador/Fachada). Esto se evidencia claramente en flujos combinados como el proceso de inicio de sesión (`login`):

- 1) **Validación de Identidad:** Invoca en primer lugar al servicio encargado de la verificación de credenciales criptográficas (`auth_service.login`), validando que la entidad del usuario sea correcta.
- 2) **Emisión de Estado:** Utilizando la misma sesión transaccional compartida dentro del objeto `uow`, delega inmediatamente en el servicio de tokens (`token_service.issue_tokens`) para generar las credenciales de acceso, compilar los metadatos planos y agendar las escrituras asíncronas en la caché.
- 3) **Garantía de Atomicidad:** Si la generación de tokens o cualquier hook posterior falla dentro del ciclo de vida del método, el bloque de contexto del servicio de aplicación aborta el proceso por completo. Esto realiza un `rollback` automático de cualquier cambio previo realizado por la validación de identidad, garantizando que el sistema jamás quede en un estado inconsistente.

E. *Desglose capa de servicios de aplicación*

La capa de servicios de aplicación actúa como el orquestador principal del sistema. No contiene lógica de negocio pura ni reglas de cálculo, sino que coordina los casos de uso dirigiendo la interacción entre las entidades del dominio, las unidades de trabajo (`UnitOfWork`) y los componentes de infraestructura externos (criptografía, generación de tokens y almacenamiento en caché).

a) *Gestión de Identidad y Mitigación de Ataques de Canal Lateral* (`AuthService`):

El servicio `AuthService` expone las operaciones básicas del ciclo de vida del usuario (`register`, `login`, `delete_account`) delegando la persistencia en la unidad de trabajo. Su diseño incorpora salvaguardas arquitectónicas críticas:

- **Inyección del Motor de Persistencia:** Cada método recibe la instancia activa de `AuthUoW`, lo que garantiza que todo el flujo del caso de uso se ejecute bajo una transacción única y controlada.
- **Mitigación de Enumeración de Usuarios (Timing Attacks):** En operaciones de autenticación o eliminación donde el usuario podría no existir, el servicio utiliza

una constante inmutable `DUMMY_HASH`. Si el repositorio devuelve un valor nulo, el sistema verifica esta credencial ficticia de todas formas. Esto asegura que el tiempo de respuesta computacional del servidor sea idéntico tanto para usuarios válidos como inválidos, bloqueando intentos maliciosos de adivinar nombres de usuario mediante análisis de latencia de red.

b) *Aplanamiento y Serialización de Contextos de Acceso* (`PermissionCompiler`):

Para maximizar el rendimiento en las capas de filtrado intermedio (**middleware**), el sistema evita consultar la base de datos relacional en cada petición HTTP. El componente `PermissionCompiler` actúa como un transformador de datos especializado:

- **Aplanamiento del Gráfico de Entidades:** Recibe la estructura relacional jerárquica del usuario (que incluye sus límites de nivel o ABAC, roles globales o RBAC y permisos relacionales acotados o ReBAC) y la transforma en un mapeo plano de cadenas de texto (`dict[str, str]`).
- **Optimización para Almacenamiento NoSQL:** Los árboles de permisos complejos y listas de roles se serializan a texto plano mediante formato JSON (`json.dumps`). Esto adecua la estructura para ser inyectada de forma directa en estructuras de datos optimizadas (como mapas de hashes) en motores de memoria intermedia distribuidos como Valkey.

c) *Orquestación del Ciclo de Vida de Sesiones* (`TokenService`):

El servicio `TokenService` centraliza la emisión, rotación y revocación de credenciales criptográficas, actuando como el puente directo entre la base de datos transaccional y la caché temporal:

- **Flujo de Emisión (`issue_tokens`):**
 - 1) Genera criptográficamente un par de tokens (acceso y actualización).
 - 2) Extrae el gráfico del usuario optimizado desde el repositorio utilizando carga ansiosa (**eager loading**).
 - 3) Registra el hash del token de actualización de forma persistente en la base de datos para auditoría.
 - 4) Compila el mapa plano de permisos y agenda un gancho post-commit transaccional (`schedule_session_cache_sync`) para escribir en Valkey con un tiempo de vida (`ttl_seconds`) idéntico a la expiración del token.
- **Rotación Criptográfica y Reautenticación (`reauth`):** Implementa el patrón de rotación de tokens. Al recibir un token de actualización válido, verifica su integridad frente al almacén relacional, invalida lógicamente el token antiguo (`is_active=False`), agenda una purga post-commit en la caché intermedia y emite un par de tokens completamente nuevo. Este flujo impide que un token interceptado sea reutilizado de forma maliciosa.

F. *Desglose capa unidad de trabajo (UoW)*

La Unidad de Trabajo es el componente encargado de centralizar y garantizar la consistencia transaccional (propiedades ACID) de la aplicación. Actúa como un motor de gestión de contexto para las operaciones asíncronas de la base de datos y la sincronización diferida de cachés de datos.

a) *El Motor Base* `UnitOfWork`:

La clase base abstracta utiliza la factoría de sesiones asíncronas de SQLAlchemy (`SessionAsync`), configurada para evitar la expiración prematura al confirmar cambios (`expire_on_commit=False`) y la descarga automática de sentencias (`autoflush=False`). Su comportamiento se rige mediante un gestor de contexto asíncrono nativo:

- **Inicio del Contexto (`__aenter__`):** Al inicializarse un bloque `async with`, el motor levanta de forma aislada una sesión activa (`AsyncSession`) e invoca inmediatamente el método abstracto `__init_repositories()`. Esto asegura que los repositorios locales se vinculen al ciclo de vida actual antes de procesar cualquier instrucción.
- **Cierre e Integridad Transaccional (`__aexit__`):** El método evalúa si el bloque de código del caso de uso terminó con éxito o error:
 - **Rollback:** Si se captura alguna excepción (`exc_type`), ejecuta un rollback automático en la base de datos relacional para revertir cualquier cambio incompleto.
 - **Commit:** Si no hay errores, confirma la transacción de forma segura mediante un commit.
- **Mecanismo de Ganchos Post-Commit (`add_post_commit_hook`):** Permite encolar llamadas asíncronas que se ejecutarán secuencialmente en el bloque `__aexit__` únicamente si la confirmación en la base de datos fue exitosa. Si ocurre un fallo en el motor transaccional primario, la cola se limpia en el bloque `finally` para prevenir estados huérfanos o desincronizados en componentes de infraestructura externos (como memorias caché o colas de mensajería).

b) *Especialización por Contextos de Negocio:*

El sistema no utiliza una única Unidad de Trabajo global; en su lugar, cada contexto delimitado (**Bounded Context**) implementa su propia variante especializada heredando de la clase base (por ejemplo, `FormDesignUoW`, `FormLogicUoW`, `GradingUoW`, entre otros). Esta especialización sigue dos reglas operativas:

- 1) **Inyección Homogénea de Sesión:** Al sobrescribir el método `__init_repositories()`, el contexto inyecta la misma instancia exacta de la sesión activa (`self.session`) en todos los repositorios requeridos para ese dominio. Esto permite coordinar operaciones heterogéneas sobre múltiples tablas distintas dentro de una única transacción indivisible.
- 2) **Control de Referencias Cruzadas:** Ciertos contextos mapean repositorios adicionales en modo de solo lectura o para verificar la integridad de las referencias antes de procesar cambios locales, optimizando la

comunicación interna sin romper el aislamiento entre dominios.

G. Desglose capa repositorios

Los repositorios aíslan el acceso a datos abstrayendo la infraestructura técnica como si se tratase de colecciones de objetos en memoria. El contrato base se define a través de tipos genéricos para estandarizar las tareas básicas de persistencia.

a) *El Repositorio Genérico Base* (`BaseRepository`): La clase abstracta `BaseRepository(Generic[ModelT])` requiere un parámetro genérico ligado a las entidades del modelo relacional (`ModelT`). Recibe la sesión activa (`AsyncSession`) gestionada por la Unidad de Trabajo correspondiente e implementa los métodos primitivos de manipulación:

- `add(entity)`: Registra la instancia de la entidad en la sesión activa del ORM.
- `delete(entity)`: Agenda la eliminación física de la entidad de la base de datos.
- `get_by_id(id)`: Resuelve consultas parametrizadas rápidas por clave primaria (`UUID`) mediante la API directa del ORM.

b) *Especialización y Optimización de Consultas:*

Cuando un agregado del dominio requiere operaciones de acceso complejas que exceden las funciones CRUD básicas, el repositorio genérico se extiende incorporando la API de selección de SQLAlchemy bajo las siguientes estrategias comunes de la arquitectura:

- **Modificaciones Lógicas frente a Eliminaciones Físicas:** Para entidades críticas o sujetas a auditorías históricas, los repositorios específicos sustituyen las sentencias destructivas por operaciones dirigidas de actualización (`update`), mutando banderas de estado (como flags de actividad) para preservar la trazabilidad forense de los datos.
- **Estrategias de Carga Ansiosa (`Eager Loading`):** Al recuperar gráficos de entidades altamente interconectadas o jerárquicas, los repositorios implementan directivas explícitas de carga masiva (`selectinload`). Esto anula el comportamiento perezoso por defecto (`lazy loading`), mitigando de raíz el problema de rendimiento de consultas redundantes de tipo $N + 1$ mediante el envío de sentencias unificadas optimizadas en una única llamada de red a la base de datos.

IV. ARQUITECTURA DE DATOS

A. Visión general de la base de datos

El sistema de persistencia del **Índice de Innovación Pública (IIP)** está diseñado como un motor dinámico, multi-inquilino (**multi-tenant**) y fuertemente auditable para la gestión, captura y evaluación de información pública.

En lugar de emplear un esquema estático de tablas rígidas para cada tipo de trámite o encuesta, la base de datos utiliza un enfoque data-driven (orientado a metadatos). Esto permite definir formularios complejos de manera completamente dinámica en tiempo de ejecución, gestionando desde la estructura

jerárquica de las preguntas hasta la validación de entrada, la lógica condicional de despliegue y los flujos de calificación.

a) *Propósito Arquitectónico del Modelo:*

A alto nivel, la arquitectura de persistencia resuelve tres grandes capacidades del sistema:

- **Definición Dinámica y Reglas de Negocio (Estructura de Formularios):** Permite modelar instrumentos de recolección jerárquicos (formularios, secciones, preguntas y campos). Integra un motor de reglas que evalúa dependencias en tiempo real (visibilidad condicional entre campos o secciones) y restricciones de validación sin requerir cambios en el código de la aplicación o en la base de datos.
- **Gobernanza, Contexto y Seguridad (Actores y Usuarios):** Vincula la identidad de los usuarios (`User`) con entidades u organizaciones del sector público (`Actor`) mediante esquemas de control de acceso granulares basados en roles globales (RBAC) y contextuales (ReBAC). Esto garantiza la segregación de datos y que cada respuesta capturada pertenezca al contexto institucional correcto.
- **Captura Polimórfica y Evaluación (Respuestas y Calificación):** Ofrece una capa de almacenamiento flexible basada en la entidad base `Answer`, capaz de persistir de forma fuertemente tipada respuestas numéricas, textuales, selecciones o archivos. Sobre estos datos capturados (`Submission`), el sistema permite aplicar rúbricas y criterios de evaluación ponderados para automatizar o estructurar la generación de resultados e índices.

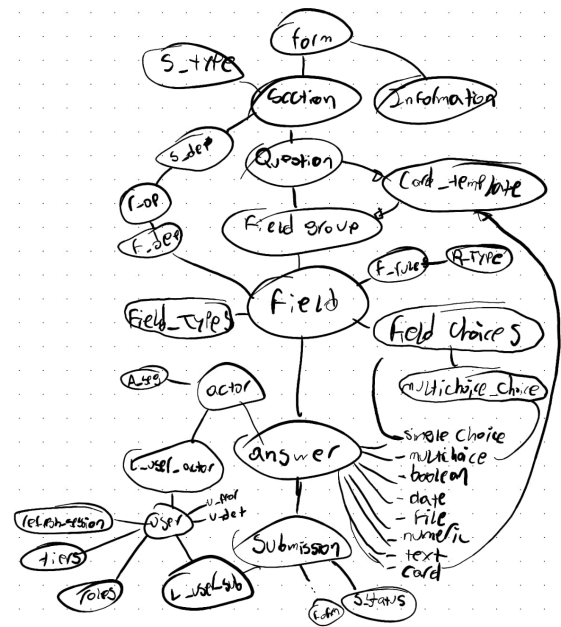


Fig. 4. Arquitectura de flujo y dependencias del sistema de persistencia.

b) Rol de SQLAlchemy y el ORM:

El uso de **SQLAlchemy** como capa de Mapeo Objeto-Relacional (ORM) abstrae la complejidad de la base de datos relacional y proporciona ventajas clave en la arquitectura:

- **Tipado Riguroso y Modelado Polimórfico**: Permite mapear la jerarquía de herencia de respuestas (`Answer` y sus especializaciones) a nivel de código de forma transparente, garantizando integridad de tipos desde PostgreSQL hasta la capa de API.
- **Gestión Módulo a Módulo mediante Schemas**: Permite mapear explícitamente cada clase Python a un esquema de PostgreSQL específico, garantizando el aislamiento lógico de dominios (autenticación, formularios, auditoría, entregas) dentro de un único motor relacional.
- **Alineación con el Ciclo de Vida y Migraciones**: Interactúa directamente con Alembic para la autogeneración e inspección del esquema de datos, garantizando que los cambios en las clases Python se reflejen de forma consistente, escalable y sin pérdida de información en los entornos de despliegue.

B. Entidades de dominio

```
Packages/shared/src/shared/models
├── actors.py
├── audit.py
├── auth.py
├── files.py
├── forms.py
├── grading.py
├── __init__.py
├── interactions.py
├── links.py
├── reference.py
├── rules.py
├── submissions.py
└── targets.py
```

El modelo de datos de la plataforma se compone de un conjunto de entidades ORM declarativas implementadas con SQLAlchemy, estructuradas de acuerdo con la organización por esquemas (**Schema-per-Domain**). A continuación, se detallan las entidades de dominio organizadas por su ámbito funcional:

a) 1. *Autenticación y Usuarios* (`auth`): Este dominio engloba los modelos responsables de la gestión de la identidad, credenciales, sesiones y atributos asociados a los usuarios del sistema:

- **User**: Entidad central que representa las cuentas de usuario en el sistema. Almacena credenciales (correo electrónico, hash de contraseña), estado de verificación, marca de tiempo de la última sesión, banderas administrativas y nivel/vínculo de cuenta (`tier_id`).
- **UserProfile**: Extensión uno a uno de la entidad `User` que contiene la información personal básica (nombre, apellidos, documento de identidad, género, fecha de nacimiento, teléfono y avatar).

- **UserDetails**: Modelo opcional asociado al usuario para almacenar información profesional, laboral o demográfica adicional.

- **RefreshSession**: Mantiene el estado de las sesiones activas y los tokens de refresco (JWT) para la autenticación persistente y segura.

- **Permission**, **SystemRole**, **ResourceRole**: Entidades pertenecientes conceptualmente al control de acceso (RBAC/ReBAC) que definen acciones atómicas (permisos globales o de recurso) y agrupaciones de roles asignables a los usuarios.

b) 2. *Actores y Grupos de Interés* (`actors`): Administra el modelo de actores organizacionales y entidades externas que interactúan con las herramientas del sistema:

- **ActorSegment**: Define categorías o clasificaciones conceptuales para agrupar actores según sus características o nivel dentro de la plataforma (por ejemplo, entidades territoriales, instituciones públicas, etc.).

- **Actor**: Representa una entidad u organización concreta que responde a formularios o evaluaciones. Registra identificadores oficiales (`sigep_code`, `treasury_code`), siglas, denominación pública, descripción y su respectiva vinculación a un `ActorSegment`.

c) 3. *Estructuración de Formularios* (`forms`): Define la arquitectura declarativa utilizada para construir instrumentos de recolección de información dinámicos e interactivos:

- **Form**: Entidad principal que representa un formulario completo. Contiene metadatos generales (código, título, descripción) y un ciclo de vida configurado por fechas de inicio, cierre e indicadores de publicación o archivado.

- **Section**: Subdivisión lógica dentro de un formulario. Soporta estructuras jerárquicas recursivas (relación padre-hijo/árbol) para organizar secciones intermedias o complejas.

- **Question**: Representa las preguntas o reactivos pertenecientes a una sección. Administra el ordenamiento, requerimiento obligatorio de respuesta y vinculación a criterios de evaluación.

- **FieldGroup** y **Field**: Definen la estructura técnica de los campos de entrada asociados a una pregunta (por ejemplo, campo de texto, numérico, opción única, archivo).

- **FieldChoice**: Opciones seleccionables asociadas a campos de tipo elección (**single choice** o **multi choice**).

- **CardTemplate**: Plantilla para gestionar grupos de campos repetibles o bucles de entrada (**loop cards**).

d) 4. *Reglas y Lógica de Dependencia* (`rules`): Almacena las reglas dinámicas y restricciones aplicadas sobre la interfaz de recolección:

- **FieldRule**: Define validaciones específicas aplicadas sobre los campos de entrada (longitud mínima/máxima, expresiones regulares, etc.).

- **FieldDependency**: Establece condicionales de visibilidad o activación entre campos según el valor de un campo previo y un operador relacional.

- **SectionDependency**: Modela dependencias dinámicas que ocultan o muestran secciones completas del formulario en función de las respuestas ingresadas.
- e) 5. *Captura y Entregas* (*submissions*): Gestiona los datos capturados y el estado de procesamiento de las respuestas enviadas por los actores:
 - **Submission**: Instancia de envío que consolida la entrega formal realizada por un **Actor** respecto a un **Form** específico, controlando su ciclo de vida a través de estados.
 - **Answer**: Entidad polimórfica base para registrar la respuesta brindada a un **Field** dentro de una entrega.
 - **Especializaciones de Answer**: Subtipos declarativos derivados según el tipo de dato recolectado (**AnswerText**, **AnswerNumeric**, **AnswerBoolean**, **AnswerDate**, **AnswerSingleChoice**, **AnswerMultiChoice**, **AnswerFile**).
 - **AnswerCardEntry**: Instancia específica que almacena datos recopilados a través de plantillas de bucles de entrada (**CardTemplates**).
- f) 6. *Evaluación y Calificación* (*grading*): Procesa el cálculo de puntuaciones, retroalimentación y resultados finales:
 - **Criterion**: Criterio o rúbrica de evaluación asociado de manera directa a una pregunta o asignación para normar la calificación.
 - **Grade**: Almacena las calificaciones cuantitativas individuales asignadas por un evaluador a las respuestas enviadas en una entrega.
 - **Result**: Registro consolidado y ponderado del puntaje final calculado para una entrega (**Submission**).
- g) 7. *Gestión de Archivos* (*files*): Administra los recursos binarios y metadatos del sistema:
 - **FileType**: Catálogo de formatos de archivo admitidos, restricciones de extensión, MIME types y límite de peso máximo.
 - **File**: Registro del archivo físico subido, conservando metadatos como el tamaño, hash de integridad y ruta de almacenamiento en disco/S3.
 - **Attachment**: Vínculo contextual que asocia un **File** con entidades del sistema (como un usuario, un actor o una respuesta) especificando visibilidad y alcance.
- h) 8. *Auditoría e Interacciones* (*audit, interactions, reference*):
 - **Log** (*audit*): Registra eventos y trazabilidad de acciones dentro de la plataforma para auditoría técnica y de seguridad.
 - **Comment** y **Notification** (*interactions*): Gestión de comentarios contextuales entre usuarios y centro de notificaciones de la plataforma.
 - **Tablas de Catálogo** (*reference*): Tablas auxiliares no modificables que definen constantes de dominio (**RelationalOperator**, **RuleType**, **SubmissionStatusType**, etc.).

C. Relaciones

El modelo relacional del sistema establece la interacción entre dominios mediante tres patrones de asociación: relaciones directas uno a muchos ($1:N$), herencia polimórfica y relaciones de muchos a muchos ($M:N$). Para estas últimas, la arquitectura prescinde de tablas intermedias implícitas o anónimas y define explícitamente entidades de enlace en el esquema **links** (**links.py**), lo que garantiza el control estricto del ciclo de vida de los datos, integridad referencial mediante claves foráneas compuestas y la posibilidad de extender metadatos en la relación.

a) 1. Control de Acceso Global (RBAC) y Contextual (ReBAC):

El motor de seguridad descompone los permisos y roles a través de dos mecanismos de asociación explícitos:

- **SystemRolePermissionLink**: Conecta la entidad **SystemRole** con **Permission**. Modela el esquema de **Role-Based Access Control** (RBAC) a nivel global, permitiendo asociar múltiples permisos atómicos a un rol de sistema. Su clave primaria es compuesta (**system_role_id**, **permission_id**) con eliminación en cascada (**ON DELETE CASCADE**).
- **ResourceRolePermissionLink**: Implementa la lógica de **Relationship-Based Access Control** (ReBAC). Asocia permisos atómicos (**Permission**) a roles acotados por recurso (**ResourceRole**), definiendo capacidades específicas que un usuario puede ejercer sobre una entidad en particular (por ejemplo, editar un formulario específico o evaluar un envío determinado).
- **UserSystemRoleLink**: Asigna roles de sistema (**SystemRole**) a un usuario (**User**). Permite que una cuenta posea múltiples roles globales dentro de la plataforma.

b) 2. Membresía y Adscripción Institucional (*UserActorLink*):

- **UserActorLink**: Es la entidad de enlace fundamental que vincula a los usuarios (**User**) con las entidades u organizaciones del sector público (**Actor**).
 - **Soporte Multitenant y Delegación**: Permite que un usuario pertenezca a uno o varios actores organizacionales.
 - **Integración con ReBAC**: Incluye de manera opcional una clave foránea hacia **ResourceRole** (**resource_role_id**). Esto permite declarar qué rol específico desempeña el usuario dentro de esa entidad en particular (por ejemplo, **Administrador de Entidad**, **Diligenciador** o **Evaluador**), acotando la visibilidad y permisos dentro del contexto funcional de dicho actor.

c) 3. Enlace de Selección Múltiple (*MultiChoiceOptionLink*):

- **MultiChoiceOptionLink**: Modela la relación $M:N$ entre las opciones de respuesta seleccionables (**FieldChoice** en el esquema **forms**) y la respuesta de opción múltiple cap-

turada (`AnswerMultiChoice` en el esquema `submissions`).

- Su clave primaria compuesta (`choice_id`, `multi_choice_answer_id`) asegura que una entrega de tipo selección múltiple pueda almacenar de forma fuertemente tipada y normalizada la lista exacta de opciones marcadas por el usuario.

d) Resumen de Entidades de Enlace (`links.py`):

A continuación se sintetiza la estructura técnica de las tablas de unión declaradas dentro del esquema `links`:

Entidad / Tabla	Esquema	Propósito y Estructura de Claves
<code>SystemRolePerm</code>	<code>links</code>	Asociación RBAC entre <code>SystemRole</code> y <code>Permission</code> (Clave primaria compuesta).
<code>ResourceRolePerm</code>	<code>links</code>	Asociación ReBAC entre <code>ResourceRole</code> y <code>Permission</code> (Clave primaria compuesta).
<code>UserSystemRole</code>	<code>links</code>	Asignación de roles globales <code>SystemRole</code> a cuentas de <code>User</code> .
<code>UserActorLink</code>	<code>links</code>	Vinculación de <code>User</code> con <code>Actor</code> , incluyendo un <code>ResourceRole</code> opcional por contexto.
<code>MultiChoiceOptions</code>	<code>links</code>	Asociación de opciones elegidas (<code>FieldChoice</code>) en una respuesta (<code>AnswerMultiChoice</code>).

D. Organización del esquema

La plataforma del **Índice de Innovación Pública (IIP)** implementa un enfoque de aislamiento y modularidad en la capa de persistencia denominado Schema-per-Domain. En lugar de centralizar todas las entidades dentro del esquema predeterminado por defecto (`public`), la arquitectura organiza las tablas de PostgreSQL en esquemas lógicos o dominios funcionales:

- **reference**: Tablas de catálogo e información estática compartida (`log_action_types`, `file_types`, `user_tiers`, `resource_roles`, `system_roles`, `permissions`, `field_types`, etc.).
- **auth**: Gestión de credenciales, sesiones y perfiles de usuario (`users`, `refresh_sessions`, `user_details`, `user_profiles`).
- **audit**: Registros de trazabilidad y logs de auditoría del sistema (`logs`).

- **files**: Almacenamiento de metadatos de archivos y adjuntos (`files`, `attachments`).
- **links**: Tablas intermedias para relaciones de autorización y membresía (`user_system_role_links`, `system_role_permission_links`, `resource_role_permission_links`).
- **interactions**: Gestión de comentarios y notificaciones (`comments`, `notifications`).
- **actors**: Administración de entidades y grupos de interés (`actors`, `actor_segments`).
- **forms**: Estructuración de formularios, secciones y preguntas (`forms`, `sections`, `questions`, `fields`, `field_groups`).
- **rules**: Reglas de dependencia y validación de formularios (`section_dependencies`, `field_rules`, `field_dependencies`).
- **submissions**: Captura de respuestas y gestión de entregas (`submissions`, `answers_*`).
- **grading**: Procesos de evaluación, asignaciones y calificaciones (`assignments`, `criteria`, `grades`, `results`).

Cada tabla se mapea explícitamente mediante una estructura inmutable `TableInfo("nombre_tabla", "esquema")`, asociando cada entidad ORM con su esquema destino.

a) Automatización de Creación mediante Introspección:

Durante la fase de inicialización o despliegue inicial del sistema (invocado mediante el script de gestión con la bandera `--setup`), el módulo de persistencia ejecuta un procedimiento automatizado para garantizar la existencia física de los esquemas requeridos antes de aplicar migraciones de datos:

- 1) **Inspección Dinámica de Clases (`get_all_schemas`)**: A través del método `inspect.getmro(TargetTable)`, el sistema analiza en tiempo de ejecución la jerarquía de herencia de clases de `TargetTable` y su clase base `CoreTargetTable`. Mediante introspección (`vars()`), evalúa cada atributo para identificar instancias de `TableInfo` y extraer de forma dinámica e iterativa el valor de la propiedad `schema`, consolidándolos en un conjunto (`set`) de nombres únicos de esquemas requeridos.
- 2) **Ejecución Transaccional Segura**: Se establece una conexión síncrona `SessionSync()` con PostgreSQL. Para cada esquema identificado en la introspección, se emite de forma iterativa la instrucción DDL mediante la clase de SQLAlchemy `CreateSchema(schema, if_not_exists=True)`.

La cláusula `if_not_exists=True` previene excepciones en el motor de base de datos (como el código de error PostgreSQL `42P06`) si la estructura ya existe.

- 3) **Manejo de Transacciones e Informes**: Cada operación sobre un esquema realiza una confirmación explícita (`session.commit()`). Si ocurre un error de tipo `SQLAlchemyError`, el flujo ejecuta un `session.rollback()` individual, añade el esquema a la lista de fallos y continúa con el procesa-

miento de los demás. Al concluir, el sistema emite un reporte detallado en los logs clasificando los esquemas creados exitosamente y aquellos que fallaron.

E. Gestión de Migraciones con Alembic

Para garantizar la evolución controlada del esquema de la base de datos sin pérdida de información, el sistema integra **Alembic** como motor de migraciones. Dentro del componente de persistencia (`Persistence/src/migrator/alembic/`), el archivo crítico que orquesta este proceso es `env.py`.

Este script es invocado internamente por Alembic en dos escenarios principales:

- **Autogeneración:** Detectar cambios en los modelos de SQLAlchemy para estructurar nuevos archivos de revisión.
- **Ejecución:** Aplicar el historial de cambios (desde la migración inicial hasta la más reciente) al levantar el entorno.

a) Mecanismo de Sincronización de `env.py`:

El script interactúa directamente con el núcleo de la aplicación mediante tres pasos fundamentales:

- 1) **Carga Dinámica de Metadatos:** Recupera el objeto `Base.metadata` que consolida todos los modelos del sistema importados desde `shared.models`. Esto permite a Alembic contrastar de forma automática la estructura declarada en código con el estado real de las tablas en la base de datos.
- 2) **Inyección de la URL de Conexión:** Sobrescribe la configuración estática del archivo `.ini` tradicional de Alembic, inyectando de forma dinámica la constante `SYNC_URL`. Esto garantiza que los comandos apunten siempre a la instancia correcta del entorno actual.
- 3) **Control de Versiones Personalizado:** Modifica la tabla nativa de seguimiento de Alembic configurando el parámetro `version_table="alembic_automatizado"`. En esta tabla se almacena el identificador único de la última revisión aplicada con éxito.

b) Modos de Operación:

Dependiendo del contexto de ejecución detectado por `context.is_offline_mode()`, el script bifurca su comportamiento en dos funciones:

- **`run_migrations_offline()`:** Configura el contexto de manera aislada utilizando únicamente la URL de conexión. Se utiliza para generar scripts SQL planos sin necesidad de abrir una conexión interactiva o directa con el motor de base de datos.
- **`run_migrations_online()`:** Es el modo por defecto ejecutado por el contenedor de persistencia. Extrae el motor de conexión (`sync_engine`), abre una transacción segura y aplica de manera secuencial todas las migraciones pendientes directamente sobre la base de datos relacional.

V. PERSISTENCIA E INTEGRIDAD DE DATOS

La capa de persistencia constituye la base de la integridad operacional de la plataforma IIP. Su diseño no se limita al

almacenamiento relacional, sino que implementa un ecosistema de gestión automatizada que garantiza la trazabilidad, consistencia y recuperabilidad de la información a lo largo de todo el ciclo de vida del dato. A través de una serie de componentes especializados —orquestados por el servicio `persist`—, el sistema gestiona desde la creación dinámica de esquemas y la evolución del modelo mediante migraciones versionadas, hasta la ingesta inteligente de volúmenes históricos y la salvaguarda de la base de datos mediante respaldos automáticos.

Este diseño modular asegura que el estado del sistema sea siempre auditable y replicable. Al desacoplar las tareas de mantenimiento de la base de datos de la lógica de negocio activa, el sistema permite operaciones de administración (como migraciones de esquema o restauración de respaldos) de forma aislada y controlada, cumpliendo con los requisitos de robustez y disponibilidad exigidos por la Veeduría Distrital.

A. Creación de DB schemas

Este módulo automatiza la creación de esquemas de base de datos en PostgreSQL durante el despliegue inicial del sistema, inspeccionando dinámicamente las clases del modelo de datos para garantizar la existencia de las estructuras necesarias. El script de persistencia extrae de manera única los nombres de los esquemas definidos en la clase `TargetTable` y sus clases base mediante introspección de código, identificando cada atributo que implemente el tipo `TableInfo`.

Posteriormente, abre una sesión síncrona con la base de datos y ejecuta de forma iterativa comandos seguros de creación de esquemas que previenen errores por duplicación. El flujo maneja de forma independiente cada confirmación o reversión ante fallos y genera un informe final en el registro del sistema detallando las estructuras creadas de manera exitosa y las fallidas.

El script `launcher.sh` ejecuta automáticamente este proceso de inicialización al detectar la bandera `--setup`, simplificando la configuración del entorno cuando el sistema se despliega por primera vez en un servidor virgen.

B. Alembic: Infraestructura y Operación

Para asegurar la persistencia y sincronización de las migraciones entre el host y el contenedor `persist`, el volumen de versiones de Alembic debe montarse en `docker-compose.yml`. Esto refleja la estructura de archivos generada en el host directamente dentro de `Persistence/src/migrator/alembic/versions`.

```
persist:
  ...
  volumes:
    # Huesped : Contenedor
    - "Persistence/src/migrator/alembic/versions:/api/migrator/alembic/versions"
    # Cambie sólo la ruta en huésped, NO cambie ruta en contenedor.
```

El montaje de este volumen es obligatorio. Si el directorio `versions` en el host está vacío o desincronizado al levantar el servicio, las migraciones automáticas fallarán al no poder contrastar los metadatos con el historial de revisiones previo administrado por el motor.

a) Generación de Nuevas Revisiones:

El sistema facilita la creación de nuevas migraciones mediante el script `launcher.sh`. Este mecanismo automatiza la detección de cambios entre la definición actual de los modelos y el estado real de la base de datos, generando los archivos de migración necesarios de forma estructurada.

Para crear una nueva revisión tras modificar los modelos, utilice la bandera `--revision` especificando una descripción breve de los cambios:

```
./launcher.sh --revision  
"add_user_bio_field"
```

El script ejecuta internamente el comando `alembic revision --autogenerate` dentro del contenedor `persister`. Esto asegura que el código generado sea coherente con el entorno de ejecución, capturando automáticamente cualquier adición, modificación o eliminación de columnas o tablas.

i Nota: Al utilizar `--revision`, el sistema crea un nuevo archivo de migración en `Persistence/src/migrator/alembic/. versions/`. Es recomendable revisar el contenido de este archivo antes de aplicar la migración en entornos de producción para asegurar que Alembic ha detectado correctamente los cambios deseados.

C. Poblado de sistema

Este módulo automatiza la carga de datos maestros e iniciales (seeds) en la base de datos tras asegurar la existencia de los esquemas y aplicar las migraciones correspondientes. El mecanismo implementado en `seeder.py` escanea dinámicamente el directorio interno `seeds/`, ordenando alfabéticamente los archivos encontrados para garantizar una secuencia de ejecución predecible y respetar las dependencias relacionales subyacentes. Utilizando el módulo `importlib.util` de Python, el script realiza una carga reflexiva de cada archivo `.py`, busca de forma explícita una función ejecutable llamada `upgrade()` y la invoca de manera aislada dentro de un bloque controlado de excepciones. El flujo captura errores individuales por archivo para evitar que un fallo en un set de datos interrumpa todo el proceso de inicialización, generando un registro detallado en el `logger` y volcando la traza completa (`traceback`) en la consola ante cualquier eventualidad.

Al igual que los módulos previos de persistencia, este componente es invocado automáticamente por el script `launcher.sh` al ejecutar la bandera `--setup` durante el despliegue en un entorno virgen, asegurando que las tablas base queden completamente pobladas.

Para asegurar el correcto orden de inserción (por ejemplo, registrar roles antes que usuarios), el directorio debe mantener una nomenclatura secuencial estricta tal como se ilustra en la siguiente estructura física:

```
Persistence/src/seeder  
├── seeder.py  
└── seeds  
    ├── 00a_seed_log_action_types.py  
    ├── 00b_seed_file_types.py  
    ├── 00c_seed_permissions.py  
    ├── 00d_seed_resource_roles.py  
    ├── 00e_seed_system_roles.py  
    ├── 01b_seed_rule_types.py  
    ├──  
    ├── 01c_seed_relational_operators.py  
    ├── 01d_seed_user_tiers.py  
    ├──  
    ├── 01f_seed_submission_status_types.py  
    ├── 10a_seed_users.py  
    ├── 30a_seed_field_types.py  
    ├──  
    ├── 33a_seed_notification_types.py  
    ├── 33b_seed_comment_types.py  
    └── __init__.py
```

i Nota: Nota: Cualquier script de inicialización nuevo que se añada a la carpeta `seeds/` debe implementar obligatoriamente la función `upgrade()`. Se recomienda seguir el patrón numérico/alfabético prefijado (`00a_`, `10a_`) para controlar de forma explícita el orden de carga y evitar fallos por restricciones de llave foránea en la base de datos.

D. Poblado de históricos

Este componente gestiona la ingesta masiva de datos históricos y estructuras complejas en el sistema a través de la API pública de los microservicios, en lugar de realizar inserciones directas en el motor de persistencia. El módulo se orquesta de manera asíncrona mediante `asyncio` y `httpx`, conectándose con el servicio de autenticación (`api_auth`) y el núcleo del sistema (`api_core`) utilizando variables de entorno para resolver los endpoints internos de la red de Docker. El flujo extrae las credenciales iniciales de un archivo TOML administrado mediante secretos de Docker (`/run/secrets/users_file`), priorizando cuentas de nivel `root` para autenticarse, obtener el token Bearer correspondiente e inyectarlo automáticamente en las cabeceras de las peticiones. Para mitigar fallas en tareas de larga duración, la capa del cliente (`ServiceClient`) implementa mecanismos de re-autenticación automática que refrescan el token de acceso si este expira a mitad de una transacción.

La extracción de la data histórica se delega a un conector dedicado (`GitHubConnector`) que descarga los conjuntos de datos crudos o estructuras estructuradas directamente desde repositorios remotos utilizando un to-

ken de acceso seguro provisto como secreto de Docker (`/run/secrets/github_token_seeds`). Cada registro recuperado pasa por una capa estricta de validación local mediante esquemas de Pydantic antes de despacharse hacia la API pública de `api_core`. La transferencia de datos se realiza tanto en registros individuales como en procesamiento por lotes que comparten el mismo grupo de conexiones para optimizar el rendimiento. Al procesar las solicitudes mediante peticiones HTTP estándar (`POST`), el sistema garantiza que toda la lógica de negocio, validaciones relacionales y disparadores de eventos del backend se apliquen correctamente a la data histórica, tal como ocurriría con la interacción regular de un usuario.

La estructura física del módulo organiza las tareas en el directorio `pops/` de forma secuencial, incluyendo scripts específicos de migración y plantillas locales en formatos estructurados (CSV y XLSX) dentro de subdirectorios de trabajo:

```
Persistence/src/populator
├── pops
│   ├── 10a_seed_sectors.py
│   ├── 10b_seed_entities.py
│   ├── 11a_seed_section_types.py
│   ├── 11b_seed_forms.py
│   ├── 11c_seed_sections.py
│   ├── 11d_seed_questions.py
│   ├── 11e_seed_loop_questions.py
│   ├── 11f_seed_card_templates.py
│   ├── 11g_seed_field_groups.py
│   ├── 11h_seed_fields.py
│   ├── 11i_seed_field_choices.py
│   └── 12a_seed_field_dependencies.py
│       ├── 12b_seed_field_rules.py
│       ├── 13a_seed_grading_criteria.py
│       ├── __init__.py
│       └── jhonatan
│           └── actors.actor_segments_template.csv
│               ├── actors.actors_template.csv
│               │   ├── Entidades.csv
│               │   └── Estructura_IIP.xlsx
│               └── populator.py
```

i Nota: Nota: A diferencia de los módulos de esquemas, migraciones y datos maestros, el proceso de población masiva histórica no se ejecuta de forma mandatoria con la bandera `--setup` en sistemas limpios si los secretos o conectores externos de GitHub no están mapeados. Este componente requiere que tanto el contenedor de autenticación como el servicio núcleo estén completamente activos, saludables y con la base de datos ya estructurada.

E. Backup y Restore

Este sistema gestiona los respaldos de la base de datos PostgreSQL mediante un volumen en Docker Compose que vincula el directorio local `./Persistence/src/backups` con la ruta interna `/backups` del servicio `persistier`.

```
persistier:
  ...
  volumes:
    - "Persistence/src/backups:/backups"
```

La administración se automatiza a través del script `launcher.sh` empleando dos banderas principales: `--backup`, que ejecuta de forma no interactiva un `pg_dump` nombrando el archivo con una marca de tiempo para evitar sobrescrituras, y `--restore`, que requiere el nombre del archivo `.sql` como argumento e inyecta la variable `PGPASSWORD` internamente para autenticar de manera automática la restauración mediante `psql`.

Dado que el proceso de restauración sobrescribe los datos actuales de la base de datos, se recomienda ejecutar un respaldo preventivo antes de realizar cualquier importación.

i Nota: Nota: Se debe asegurar que el directorio local `./Persistence/src/backups` cuente con los permisos de lectura y escritura correctos para que el contenedor pueda almacenar los archivos.

La gestión de respaldos se centraliza en el script de automatización utilizando las siguientes banderas:

a) *Crear un Respaldo (backup):*

Genera una copia de seguridad en caliente de la base de datos PostgreSQL utilizando la herramienta `pg_dump`. El comando se ejecuta de forma no interactiva (`-T`) y almacena el archivo resultante usando una marca de tiempo (`TIMESTAMP`) para evitar la sobrescritura de respaldos anteriores.

```
--backup)
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
BACKUP_DIR="./Persistence/src/backups"

mkdir -p "$BACKUP_DIR"

echo "Creating database backup inside $BACKUP_DIR..."

if docker compose exec -T db sh -c "pg_dump -U \"${USER_VAR}\" -d \"${DB_VAR}\" > /backups/db_backup_${TIMESTAMP}.sql"; then
    echo "✅ Backup completed successfully: $BACKUP_DIR/db_backup_${TIMESTAMP}.sql"
else
    echo "❌ Backup failed."
```

```
fi
;;
```

b) Restaurar un Respaldo (restore):

Permite restaurar un archivo .sql específico cargándolo directamente en el motor de la base de datos mediante psql. El script requiere el nombre del archivo como parámetro e inyecta la variable `PGPASSWORD` de forma interna para realizar la autenticación automática sin solicitar credenciales en la terminal.

```
--restore)
INPUT_FILE="${1:-}"
if [ -z "$INPUT_FILE" ]; then
    echo "Usage: ./launcher.sh --
restore <backup_filename.sql or path>"
    echo "Example: ./launcher.sh --
restore db_backup_20260618_232418.sql"
    exit 1
fi

FILE_NAME=$(basename
"$INPUT_FILE")

echo "Restoring database from
$FILE_NAME..."

# Inject PGPASSWORD so Postgres
authenticates automatically without
prompting
if docker compose exec -T db sh -c
"PGPASSWORD=\`${PGPASSWORD}\`"
psql -h db -U \`${USER_VAR}\` -d
\`${DB_VAR}\` < /backups/$FILE_NAME";
then
    echo "✅ Restore completed
successfully."
else
    echo "❌ Restore failed. Make
sure $FILE_NAME exists in your backups
directory."
fi
;;
```

VI. AUTENTICACIÓN Y AUTORIZACIÓN

El servicio de autenticación actúa como el pilar de seguridad de la plataforma. Su función es establecer una identidad verificable y un contexto de seguridad **stateless** que acompaña cada solicitud hacia el servicio Core, garantizando escalabilidad y resistencia frente a accesos no autorizados.

```
claims = {
  "sub": str(user_id),
  "username": str(username),
  "token_type":
str(token_type),
  "iat":
int(now.timestamp()),
```

```
"exp":
int(exp.timestamp()),
}

jti: UUID | None = None
if token_type == "refresh":
    jti = UUID(str(uuid7()))
    claims["jti"] = str(jti)
```

A. Arquitectura de Identidad y Acceso

El sistema gestiona la seguridad a través de cuatro dimensiones que operan de forma desacoplada para asegurar un control granular:

- 1) **Identidad (Auth Service):** Gestión centralizada de credenciales mediante `Argon2id` y emisión de tokens.
- 2) **Capacidad Operativa (UserTier):** Define los límites de recursos (cuotas de almacenamiento, límites de API) que el sistema aplica al usuario.
- 3) **Gobernanza Funcional (RBAC):** Definición de roles (ej. `Editor`, `Grader`) que determinan las capacidades técnicas del usuario.
- 4) **Alcance (ReBAC):** Vinculaciones dinámicas en tablas de enlace (`UserActorLink`, `UserSubmissionLink`) que definen sobre qué objetos específicos se aplican los roles.

B. Autenticación basada en JWT

La plataforma utiliza JSON Web Tokens (JWT) firmados con **ED25519**. La firma criptográfica garantiza la inalterabilidad; cualquier modificación invalida el token, siendo verificado por cada microservicio mediante la clave pública.

El token inyecta un objeto de contexto que define las capacidades operativas y funcionales del usuario, evitando consultas constantes a la base de datos:

```
claims = {
  "sub": str(user.id),
  "context": {
    "tier": "PREMIUM",
    "assignments": [
      {"entity_id": "uuid-1",
"role": "editor"},
      {"entity_id": "uuid-2",
"role": "grader"}
    ]
  },
  "iat": int(now.timestamp()),
  "exp": int(exp.timestamp()),
}
```

C. Autorización: Del RBAC al ReBAC

La autorización no es estática; sigue un modelo de Autorización por Relaciones (ReBAC):

- **Roles Contextuales:** Un usuario no posee un rol único global. Sus privilegios dependen de su relación con un objeto, almacenada en tablas de vinculación.

- **Asignación Dinámica:** El servicio Core valida que el `entity_id` solicitado en una petición coincida con una asignación legítima del usuario dentro del JWT.
- **Gestión de Concurrencia:** Para escenarios de edición simultánea, el sistema implementa bloqueo optimista mediante versionado, asegurando que los cambios se apliquen solo si el usuario posee la versión más reciente del dato.

D. Sesiones y Revocación

El ciclo de vida de la sesión se gestiona mediante JTI (JWT ID) basados en **UUIDv7**.

a) **Revocación Granular::** El JTI permite al Auth Service mantener un seguimiento preciso de las sesiones en la base de datos (RefreshSession), facilitando la revocación inmediata por usuario o dispositivo.

b) **Transporte Seguro::**

- **Clientes Web:** Implementación de cookies con directivas `HttpOnly`, `Secure` y `SameSite=Strict` para mitigar ataques XSS y CSRF.
- **Clientes Mobile:** Transporte vía cabeceras personalizadas (`X-Refresh-Token`), con validación estricta para evitar la suplantación de plataforma.

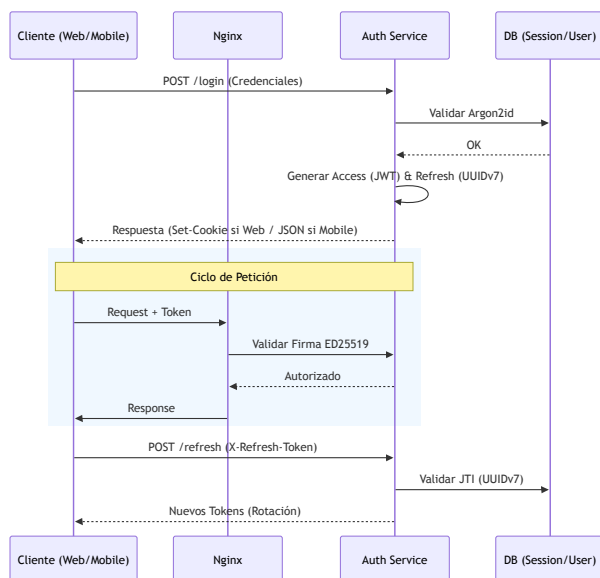


Fig. 5. Flujo de autenticación, validación y rotación de tokens.

E. Seguridad de Contraseñas

El sistema utiliza el algoritmo `Argon2id`, estándar actual de OWASP. Este enfoque garantiza máxima resistencia contra ataques de fuerza bruta al combinar un salting único por usuario con parámetros de configuración de memoria (memory-hard), lo cual neutraliza la efectividad de intentos de descifrado mediante hardware especializado como GPUs.

BIBLIOGRAFÍA

[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.

[2] Veeduría Distrital, «Índice de Innovación Pública (IIP) Platform». [En línea]. Disponible en: <https://github.com/>

[3] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[5] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

[6] V. Vernon, *Implementing Domain-Driven Design*. Addison-Wesley, 2013.

[7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[8] N. Ford, M. Richards, P. Sadalage, y Z. Dehghani, *Software Architecture: The Hard Parts*. O'Reilly Media, 2021.

[9] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.

[10] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures», Doctoral dissertation, 2000.

[11] S. Ramírez, «FastAPI». [En línea]. Disponible en: <https://fastapi.tiangolo.com/>

[12] M. Bayer, «SQLAlchemy 2.0 Documentation». [En línea]. Disponible en: <https://sqlalchemy.org/>

[13] PostgreSQL Global Development Group, «PostgreSQL Documentation». [En línea]. Disponible en: <https://postgresql.org/>

[14] M. Bayer, «Alembic Documentation». [En línea]. Disponible en: <https://alembic.sqlalchemy.org/>

[15] Docker Inc., «Docker Documentation». [En línea]. Disponible en: <https://docker.com/>

[16] NGINX, Inc., «NGINX Documentation». [En línea]. Disponible en: <https://nginx.org/>

[17] S. Colvin, «Pydantic Documentation». [En línea]. Disponible en: <https://pydantic.dev/>

[18] Python Software Foundation, «Python 3 Documentation». [En línea]. Disponible en: <https://python.org/>

[19] M. Jones, J. Bradley, y N. Sakimura, «JSON Web Token (JWT)», RFC 7519, 2015. doi: 10.17487/RFC7519.

[20] S. Josefsson y I. Liusvaara, «Edwards-Curve Digital Signature Algorithm (EdDSA)», RFC 8032, 2017. doi: 10.17487/RFC8032.

[21] A. Biryukov, D. Dinu, y D. Khovratovich, «Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications», RFC 9106, 2021. doi: 10.17487/RFC9106.

ÍNDICE

I)	Introducción	1
I.A)	Propósito	1
I.B)	Alcance	1
I.C)	Audiencia destinada	2
I.D)	Visión general de la plataforma	2
I.E)	Principios de diseño	2
II)	Arquitectura del sistema	3
II.A)	Arquitectura de alto nivel	3
II.B)	Stack tecnológico	3
II.C)	Arquitectura de despliegue (Docker)	4
II.D)	Variables de entorno	4
II.E)	Arquitectura de volúmenes	5
II.E.a)	1. Volúmenes Nombrados (Named Volumes)	5
II.E.b)	2. Montajes de Directorio (Bind-Mounts)	5
II.F)	Gestión de secretos	5
II.G)	Comunicación entre servicios	5
II.H)	Estrategia de construcción de imágenes	6
II.I)	Librería compartida	6
III)	Arquitectura de dominio	6
III.A)	Diseño guiado por el dominio (DDD)	7

III.B)	Ciclo de vida de la solicitud	7	IV.C.d)	Resumen de Entidades de Enlace (<code>links.py</code>)	13
III.C)	Desglose capa API	7	IV.D)	Organización del esquema	13
III.C.a)	Aislamiento y Delegación de Casos de Uso	7	IV.D.a)	Automatización de Creación mediante Introspección	13
III.C.b)	Gestión del Contexto de Transporte de Sesión (<code>SessionContext</code>)	8	IV.E)	Gestión de Migraciones con Alembic	14
III.C.b.a)	Seguridad en Tipado y Serialización	8	IV.E.a)	Mecanismo de Sincronización de <code>env.py</code>	14
III.D)	Desglose capa orquestrador	8	IV.E.b)	Modos de Operación	14
III.D.a)	Gestión del Límite Transaccional	8	V)	Persistencia e integridad de datos	14
III.D.b)	Coordinación de Casos de Uso Multidominio	8	V.A)	Creación de DB schemas	14
III.E)	Desglose capa de servicios de aplicación	8	V.B)	Alembic: Infraestructura y Operación	14
III.E.a)	Gestión de Identidad y Mitigación de Ataques de Canal Lateral (<code>AuthService</code>)	8	V.B.a)	Generación de Nuevas Revisiones ...	15
III.E.b)	Aplanamiento y Serialización de Contextos de Acceso (<code>PermissionCompiler</code>)	9	V.C)	Poblado de sistema	15
III.E.c)	Orquestación del Ciclo de Vida de Sesiones (<code>TokenService</code>)	9	V.D)	Poblado de históricos	15
III.F)	Desglose capa unidad de trabajo (UoW)	9	V.E)	Backup y Restore	16
III.F.a)	El Motor Base <code>UnitOfWork</code>	9	V.E.a)	Crear un Respaldo (backup)	16
III.F.b)	Especialización por Contextos de Negocio	9	V.E.b)	Restaurar un Respaldo (restore)	17
III.G)	Desglose capa repositorios	10	VI)	Autenticación y Autorización	17
III.G.a)	El Repositorio Genérico Base (<code>BaseRepository</code>)	10	VI.A)	Arquitectura de Identidad y Acceso	17
III.G.b)	Especialización y Optimización de Consultas	10	VI.B)	Autenticación basada en JWT	17
IV)	Arquitectura de datos	10	VI.C)	Autorización: Del RBAC al ReBAC	17
IV.A)	Visión general de la base de datos	10	VI.D)	Sesiones y Revocación	18
IV.A.a)	Propósito Arquitectónico del Modelo	10	VI.D.a)	Revocación Granular:	18
IV.A.b)	Rol de SQLAlchemy y el ORM	11	VI.D.b)	Transporte Seguro:	18
IV.B)	Entidades de dominio	11	VI.E)	Seguridad de Contraseñas	18
IV.B.a)	1. Autenticación y Usuarios (<code>auth</code>)	11	bibliografía		18
IV.B.b)	2. Actores y Grupos de Interés (<code>actors</code>)	11	library.bib		
IV.B.c)	3. Estructuración de Formularios (<code>forms</code>)	11			
IV.B.d)	4. Reglas y Lógica de Dependencia (<code>rules</code>)	11			
IV.B.e)	5. Captura y Entregas (<code>submissions</code>)	12			
IV.B.f)	6. Evaluación y Calificación (<code>grading</code>)	12			
IV.B.g)	7. Gestión de Archivos (<code>files</code>) ..	12			
IV.B.h)	8. Auditoría e Interacciones (<code>audit</code> , <code>interactions</code> , <code>reference</code>) ..	12			
IV.C)	Relaciones	12			
IV.C.a)	1. Control de Acceso Global (RBAC) y Contextual (ReBAC)	12			
IV.C.b)	2. Membresía y Adscripción Institucional (<code>UserActorLink</code>) ..	12			
IV.C.c)	3. Enlace de Selección Múltiple (<code>MultiChoiceOptionLink</code>) ...	12			